Stratégie de compteurs et durcissement du contrat FDM

Date : 2026-10-03 **Périmètre :** Stratégie de numérotation séquentielle utilisée par le POS, mécanisme de déduplication par le FDM, et forme du payload sur le câble.

Le Manuel de certification SCE 2.0 exige du POS qu'il (a) déclare sa stratégie de compteurs dans le formulaire de demande de certification et (b) démontre qu'il ne peut pas produire de trou dans la séquence en exploitation normale ni récupérer de manière incorrecte d'une erreur FDM. Le présent document est la trace officielle des deux.

---

1. Stratégie : par type d'événement, mono-terminal

La spec POS v2.2 §385-388 autorise quatre stratégies :

1. Distincte par type d'événement.
2. Continue générale tous types confondus.
3. Continue générale par terminal.
4. Distincte par type d'événement, par terminal.

Nous utilisons la #1 - distincte par type d'événement. Le MVP étant mono-terminal, le qualificatif "par terminal" est sans objet aujourd'hui ; si un mode multi-terminal est ajouté ultérieurement, la stratégie est rétro-compatible parce que la clé de doublon FDM (cf. §3) discrimine déjà par `terminalId`.

Pourquoi cette stratégie

- **Traçabilité maximale.** Un saut sur le compteur N est significatif ("une vente a été manquée") et isolé de l'activité S/F/P/R. Avec la stratégie #2, un événement non-fiscal (ouverture tiroir par exemple) et une vente partagent la même séquence monotone, brouillant la piste d'audit.
- **Wrap sûr.** Les compteurs par label se bouclent à 999 999 999 (max POS v2.2). Au volume d'un restaurant type (~150 tickets/jour), le compteur N mettrait ~18 000 ans à boucler. Les autres labels sont bornés similairement.
- **Récupération indépendante.** Un bug de gestion sur un label (par exemple un flux de remboursement défectueux corrompant C) ne peut pas se propager à un autre label.

Implémentation

`crates/pos-fiscal/src/counter.rs::FiscalCounter::next(conn, label)` implémente un UPSERT SQLite transactionnel sur la table `fiscal_counters` :

```
CREATE TABLE IF NOT EXISTS fiscal_counters (
    event_label TEXT PRIMARY KEY,
    current_value INTEGER NOT NULL DEFAULT 0
);

INSERT INTO fiscal_counters (event_label, current_value)
VALUES (?1, 1)
ON CONFLICT(event_label)
DO UPDATE SET current_value =
    CASE WHEN current_value >= 999999999 THEN 1
         ELSE current_value + 1
    END;
```

Onze tests unitaires dans `counter.rs` couvrent : démarrage à 1, incrément monotone, indépendance par type d'événement, `current()` reflète le dernier `next()`, sémantique table vide, l'invariant peek/next (`peek_next` ne modifie rien et annonce le prochain `next`), séquence stricte sans trou, rebouclage à 999 999 999, séquences indépendantes pour chaque label, et persistance entre deux connexions au même fichier.

Cette stratégie est verrouillée et NE DOIT PAS changer post-certification. Au titre de l'article 109 de la procédure de certification, un changement de stratégie nécessite un nouveau cycle de soumission.

---

2. JSON canonique sur le câble

Module : `crates/pos-fiscal/src/canonical.rs`. **Utilisé par :** `fdm_client::FdmClient::execute_raw` pour chaque corps de requête sortant.

Règles :

- Clés d'objet triées par codepoint Unicode à chaque niveau.
- Aucun espace insignifiant.
- Normalisation `serde_json` par défaut des nombres (forme la plus courte).
- Ordre des tableaux préservé (les séquences sont sémantiques).

Pourquoi envoyer du JSON canonique

Le FDM effectue sa propre canonicalisation pour la signature, donc notre choix de sérialisation N'AFFECTE PAS la signature numérique qu'il retourne. Nous canonicalisons néanmoins parce que :

1. **Octets de requête déterministes** - deux encaissements avec des entrées identiques produisent des corps de requête bit-à-bit identiques, ce qui rend les diff de journaux propres.
2. **Débogage de la clé de doublon** - quand un doublon est détecté, la spec compare champ par champ, mais disposer d'octets canoniques reproductibles permet de calculer et journaliser un hash de requête stable pour recoupement.
3. **Renvoi à l'identique** - la requête stockée dans `pending_signs` (§5) est celle produite par `canonical::to_canonical` : renvoyée telle quelle, elle satisfait la règle de doublon du FDM.

Vérification

8 tests unitaires dans `canonical.rs::tests` couvrent :

Propriété	Test
Clés top-level triées	`canonical_sorts_top_level_keys`
Invariance à l'ordre des champs	`canonical_is_field_order_invariant`
Clés d'objets imbriqués triées	`canonical_sorts_nested_objects`
Ordre des tableaux préservé	`canonical_preserves_array_order`
Déterministe sur appels répétés	`canonical_is_deterministic`
Pas d'espace dans la sortie	`canonical_no_whitespace_in_output`
Clés Unicode sans panique	`canonical_handles_unicode_keys`
Test propriété 50 inputs pseudo-random	`canonical_property_random_inputs_deterministic`

---

3. Clé de doublon FDM et renvoi à l'identique

Type : `crates/pos-fiscal/src/inputs.rs::IdempotencyKey<'a>`.

Selon la spec FDM (règle des requêtes en double), le FDM compare le quintuplet :

```
{ posId, posDateTime, terminalId, eventLabel, posFiscalTicketNo }
```

- **Quintuplet identique, contenu identique :** le FDM renvoie sa réponse déjà signée avec l'avertissement `DUPLICATE_REQUEST` et **n'avance pas** ses compteurs internes.

- **Quintuplet identique, contenu différent :** le FDM **refuse** (`INVALID_REQUEST`). Le POS ne construit jamais une nouvelle requête avec un numéro déjà envoyé : c'est pourquoi il renvoie la requête **octet pour octet** (voir ci-dessous).
- **Quintuplet différent :** le FDM traite comme un nouvel événement et signe.

Le POS n'invente donc plus de "réessai avec le même numéro" : une requête qui n'a pas reçu de réponse est conservée telle qu'elle a été envoyée, puis renvoyée à l'identique (table `pending_signs`, migration 10, voir *SCHEMA_BD_FR.md*). L'événement est archivé une seule fois, sous son premier numéro, avec ses effets (commande clôturée, paiements, livraison du ticket numérique, rapport enregistré et période fermée pour un Z, etc.).

Nous fournissons une méthode sur chaque input spécifique d'événement (`SaleInput::idempotency_key()`, `OrderInput::idempotency_key()`, ...) pour que les appelants n'aient pas à se souvenir du mapping label correspond à input. Six tests unitaires (`inputs.rs::idempotency_tests`) couvrent le contrat d'égalité et le mapping de label par type.

La clé expose aussi `render()` pour des lignes de log grep-friendly : `posId=CPOS01,posDt=2026-10-03T10:00:00+02:00,terminalId=T1,label=N,ticketNo=42`.

---

4. Blocage opérationnel sur déconnexion

Implémenté dans `crates/pos-engine/src/commands/checkout.rs` : la vente (`do_checkout`, `do_checkout_split`) passe par `checkout::sell`, qui signe par `signing::sign`. Quand le FDM n'est pas joignable, `FdmError::outcome()` renvoie `SignOutcome::NotSent` : la ligne `pending_signs` passe à `not_sent`, la commande d'encaissement renvoie `success: false` avec l'erreur « FDM not connected - cannot complete transaction » et **ne valide pas la commande** (aucune écriture métier, aucun compteur n'avance - voir §5). Il n'y a pas de pré-contrôle de statut avant la vente : c'est la signature elle-même qui échoue.

Il n'existe aucun mode hors-ligne, dans aucune build : le développement et les tests utilisent le FDM simulé (`pos-fdm-mock`), jamais un contournement du FDM.

Statuts dégradés du FDM

Le FDM expose aussi des drapeaux de dégradation opérationnelle via la query `status` (`errors[]`, `warnings[]`, `bufferCapacityUsed`). Ils sont pris en charge à deux niveaux :

- **Moteur et interface.** Le moteur (`crates/pos-engine/src/background.rs`, `commands/fdm_monitor.rs`) interroge le statut toutes les 30 s tant qu'une session est ouverte (recul jusqu'à 5 min si le FDM ne répond pas) et classe la réponse. Un FDM injoignable, ou un code bloquant (`BUFFER_FULL`, `FDM_LOCKED`, `FDM_NOT_OPERATIONAL`, `UNKNOWN_POS`), donne `connected = false` avec la gravité « bloquante », que l'interface reçoit par l'événement `fdm-status` : voyant rouge et voile qui couvre les écrans de vente. Les autres codes donnent un avertissement affiché (message FR de `ui/src/utils/fdmMessages.ts`).
- **Signature.** Même si l'interface ne bloquait pas, le FDM refuse lui-même de signer dans ces états (`BUFFER_FULL`, `FDM_LOCKED`, `FDM_NOT_OPERATIONAL`, `UNKNOWN_POS` sont dans la liste des refus de `FdmError::outcome()`, §5 point 4) : la vente échoue, rien n'est archivé, le numéro reste libre.

---

5. Compteur sans trou et sans doublon

Problème historique (B3, 2026-05-14) : `FiscalCounter::next()` était appelé **avant** la mutation FDM. Si l'appel échouait, le compteur local avait déjà avancé et l'encaissement suivant produisait un trou. Correctif B3 : `FiscalCounter::peek_next` lit le prochain numéro sans écrire, et le compteur n'avance qu'après la signature du FDM.

Deuxième durcissement : un seul verrou de signature. Le correctif B3 laissait deux risques : deux opérations simultanées pouvaient lire le même numéro, et une réponse perdue (le FDM a signé, le POS n'a

rien reçu) pouvait laisser un événement signé sans archive. Le flux est désormais le suivant (`crates/pos-engine/src/commands/signing.rs`) :

1. **Verrou global.** Toute opération qui signe un événement FDM (vente N, remboursement, commande de table P, pré-addition P, rapports X, X utilisateur et Z, copie C, entrée/sortie de travail S, mouvements d'argent et tiroir F, facture I) s'exécute sous un seul verrou (`tokio::sync::Mutex`). Le numéro de ticket est lu (`peek_next`) et la période d'ouverture choisie **à l'intérieur** du verrou : deux opérations ne partagent jamais un numéro, et un Z attend une vente en cours puis la compte. Une action de l'écran attend le verrou au plus 30 s, puis reçoit le code `signing_busy`.
2. **Requête stockée avant envoi.** La requête exacte, les événements à archiver et les effets métier sont écrits dans `pending_signs` (SQLite `synchronous=FULL`) avant tout envoi. Une seule ligne peut être ouverte à la fois.
3. **Envoi au FDM**, puis, sur succès uniquement, dans une seule transaction SQLite : avance du compteur (`FiscalCounter::next`), `archive::archive_event` (ligne fiscale), effets métier (commande clôturée, paiements, livraison, rapport, période fermée), et ligne `pending_signs` passée à `signed`. Un Z de rapport chiffre d'affaires et le Z utilisateur sont une seule requête et deux numéros (M184, `ticket_count = 2`).
4. **Classement de l'échec** (`FdmError::outcome()`) :
   - *Non envoyé* (FDM injoignable, certificat client refusé, erreur de sérialisation) : ligne `not_sent`, le numéro est libre.
   - *Refusé* (erreurs FDM toutes parmi `BUFFER_FULL`, `FDM_LOCKED`, `UNAUTHORIZED`, `INVALID_REQUEST`, `FDM_NOT_OPERATIONAL`, `UNKNOWN_POS`, `CLIENT_CERT_EXPIRED`) : ligne `refused`, le numéro est libre.
   - *Inconnu* (délai dépassé, connexion coupée, HTTP 500, corps illisible, réponse incohérente `FDM_RESPONSE_MISMATCH`, autre code) : la ligne **reste ouverte**. Le FDM a peut-être signé.
5. **Renvoi d'une réponse perdue.** Tant qu'une ligne est ouverte, chaque opération de signature commence par renvoyer la requête stockée, octet pour octet. Si le FDM l'avait signée, il renvoie sa réponse avec `DUPLICATE_REQUEST` : l'événement est archivé une fois, sous son premier numéro, avec ses effets. Si le renvoi est refusé, la ligne passe à `refused`, son numéro est libre et l'opération en cours continue. S'il n'y a toujours pas de réponse, l'opération est refusée avec `FDM error: FDM_ANSWER_PENDING|label=X|ticket=N|cause=...` (message à l'écran : "La caisse attend la réponse de la boîte noire pour l'opération précédente. Vérifiez la connexion et réessayez."). La tâche de fond (toutes les 60 s) renvoie aussi la requête : la caisse se débloque seule dès que le FDM répond. Il n'y a pas d'abandon automatique : une requête sans réponse bloque la signature jusqu'à la réponse du FDM (même principe que "pas de FDM, pas de vente"). Une panne de la base locale après une signature réussie laisse aussi la ligne ouverte : l'opération suivante l'archive.
6. **Z automatique.** `report::close_overdue_day` (tick toutes les 60 s et une fois au démarrage) saute son tour si la signature est occupée, et revérifie sous le verrou que la journée est toujours en retard : un Z manuel suivi du Z automatique ne produit jamais un second Z vide.
7. **Garde de période.** Le déclencheur `fiscal_events_period_open` refuse d'archiver un événement dont le `bookingPeriodId` désigne une période déjà fermée (code `period_closed`).

Tant qu'une vente attend la réponse du FDM, sa commande ne peut pas être modifiée (`order_sign_pending`) ; une commande web reste à l'état fiscal `signing`. Un mouvement d'argent répété, ou une même opération dont la première tentative a reçu sa réponse tardivement, n'est pas enregistré deux fois.

Tests : - `crates/pos-engine/tests/spf_e2e_no_gap.rs` : un échec FDM (500) ne fait pas avancer le compteur et n'archive rien d'inconnu (à lire avec les tests ci-dessous pour le cas "réponse perdue"). - `crates/pos-engine/tests/spf_e2e_signing_serialized.rs` : 14 tests (courses et réponses perdues) contre `pos-fdm-mock` derrière un relais qui perd les réponses (`crates/pos-engine-testkit/src/fdm_drop.rs`).

Risques résiduels : - Un FDM qui aurait perdu sa mémoire des doublons signerait le renvoi comme un nouvel événement. Les compteurs le détectent, le POS le journalise, mais ne peut pas l'empêcher. - Une requête qui reste sans réponse bloque la signature jusqu'à la réponse du FDM ou l'intervention d'un technicien (procédure dans *MANUEL_INSTALLATION_FR.md* et *MANUEL_UTILISATEUR_FR.md*). - Une action de l'écran peut attendre jusqu'à 30 s derrière un Z automatique ou un renvoi.

Voir aussi `DECLARATION_FONCTIONS_INTERDITES_FR.md` §4.

---

6. Rapports et périodes d'ouverture

Le POS v2.2 §2.2.2 introduit `bookingPeriodId` : un identifiant qui regroupe les événements d'une période de service pour les rapports. Deux points touchent directement la stratégie de compteurs décrite ci-dessus.

Les rapports lisent l'archive fiscale, jamais les compteurs

`crates/pos-fiscal/src/report_totals.rs` calcule intégralement les totaux X/Z/utilisateur (departments, payments, negQuantities, vats, `users[].totalAmount`) à partir des `fiscal_events` déjà archivés et signés de la période d'ouverture en cours (table `booking_periods`, migration `003` - voir *SCHEMA_BD_FR.md* §3.5), jamais des tables mutables `orders`/`order_lines`/`payments`. Le compteur `fiscal_counters` (§1 ci-dessus) sert uniquement à numéroter le prochain événement à envoyer au FDM ; il n'est jamais relu pour calculer un total.

Une période d'ouverture court du premier événement suivant un Z jusqu'au Z suivant, et porte la date d'ouverture (fuseau Europe/Brussels). Elle est persistée pour qu'un redémarrage du POS ne coupe jamais la journée en deux périodes : au démarrage, `commands::booking_period` reprend la période ouverte si elle existe, sinon en ouvre une nouvelle (`pos_db::queries::booking::current_period`, identifiant en UUID v4). Au plus une période est ouverte à la fois (index unique partiel) et une période close ne peut plus être modifiée ni supprimée (triggers SQL).

Le mode formation avance le compteur T, jamais R

Chaque rapport (X, Z, X utilisateur, Z utilisateur) est archivé et compté sous `EventLabel::R.in_mode(training_mode)` (`crates/pos-fiscal/src/types.rs`), qui bascule sur `T` en formation - exactement le même mécanisme "distinct par type d'événement" que pour les ventes (§1). Un Z de formation avance donc le compteur T, jamais R : il ne consomme aucun numéro de Z. Pour la même raison, `commands::report::do_generate_z_report` saute explicitement l'enregistrement dans `z_reports` et la clôture de la période d'ouverture quand `config.training_mode` est vrai - un Z de formation est un exercice qui ne clôture ni la comptabilité, ni le service en cours.

---

7. Reproductibilité

```
cd pili-pos
cargo test --package pos-fiscal --lib counter
cargo test --package pos-fiscal --lib canonical
cargo test --package pos-fiscal --lib idempotency_tests
cargo test --package pos-engine --test spf_e2e_no_gap
```

Totaux : totaux par binaire de test dans *RAPPORT_TESTS_BASELINE_FR.md*, régénérés par `scripts/generate_test_rep`